Praktikum 1 - Matakuliah Pilihan Web (API Development)
Program Studi: Teknik Informatika

Lakukan praktikum dibawah ini, dan buat screenshot untuk pembuktian mengerjakan setiap poin dengan mengisi tabel dibawah, kemudian tunjukan hasil akhir dari men-share repository github yang telah dibuat.

A. Setup Laravel dan Mysql

1. Buat project laravel baru dengan composer dengan nama laravel_api (Gunakan versi 12.x atau yang lebih baru)
2. Mempersiapkan database dan buat database baru bernama dblaravelapi
3. Update .env dan hubungkan dengan nama database dblaravelapi
4. Lakukan migrasi dengan menggunakan php artisan migrate
5. Pastikan laravel sudah terhubung dengan tabel dengan mengecek tabel yang dibuat laravel dalam mysql

B. Membuat Controller

1. Buatlah sebuah controller dengan nama Api/ProductController
   php artisan make:controller Api/ProductController
   (Pastikan ada di dalam folder Api)
   Pada bootstrap/app.php tambahkan setelah code web: __DIR
   api: __DIR__.'/../routes/api.php',

```
return Application::configure(basePath: dirname(__DIR__))
    ->withRouting(
        web: __DIR__.'/../routes/web.php',
        api: __DIR__.'/../routes/api.php',
        commands: __DIR__.'/../routes/console.php',
        health: '/up',
```

2. Tambahkan Routes baru di router/api.php
   Route::get('/products', [ProductController::class, 'index']);
   (Apabila file api.php belum ada, clone web.php dan hapus routes "/")
3. Test API dengan mengunjungi localhost:8000/api/products

C. Gunakan database dan Model

1. Pastikan database sudah terkoneksi dengan laravel.
2. Buatkan sebuah tabel dengan nama tblproducts dengan field (id, nama, deskripsi, foto, dan harga) untuk tipe data menyesuaikan.

3. Tambahkan minimal 10 data untuk produk dengan nama2 product elektronik seperti tv, kulkas, ac, dan lain sebagainya.
4. Buat model bernama Product
   php artisan make:model Product
5. Hubungkan model Product dengan tblproduct dengan mengisi pada class Product

```php
class Product extends Model
{
    protected $table = 'tblproducts';
    protected $primaryKey = 'id';
    public $timestamps = false;

    // Field yang boleh diisi
    protected $fillable = [
        'nama',
        'deksripsi',
        'foto',
        'harga',
    ];

}
```

6. Pada Product controller ubah dalam fungsi index, dapatkan data dari model Product
   Dengan menambahkan $products = Product:all();
   Dan lakukan return data => $products

D. Membuat CRUD standar API Resource

1. Ubah api.php dengan api Resource yang sesuai standar laravel

```php
use Illuminate\Support\Facades\Route;
use App\Http\Controllers\Api\ProductController;

// Route::get('/products', [ProductController::class, 'index']);

Route::apiResource('products', ProductController::class);
```

Dengan apiResource kamu tidak perlu menambahkan routes satu persatu untuk ProductController seperti create, update, delete, search dan view. Namun ada syarat penamaan fungsi agar routes ini bekerja yaitu **index, show, store, update, dan destroy.**

2. Ubah ProductController untuk fungsi-fungsi berikut **index, show, store, update, dan destroy.**

```php
// 🔹 GET /products
public function index()
{
    $products = Product::all();

    return response()->json([
        'status' => true,
        'data' => $products
    ]);
}
```

Gambar 04: Fungsi Index pada ProductsController

```php
// 🔹 GET /products/{id}
public function show($id)
{
    $product = Product::find($id);

    if (!$product) {
        return response()->json([
            'status' => false,
            'message' => 'Product tidak ditemukan'
        ], 404);
    }

    return response()->json([
        'status' => true,
        'data' => $product
    ]);
}
```

Gambar 05: Fungsi Show pada ProductsController

```php
// 🔹 POST /products
public function store(Request $request)
{
    $request->validate([
        'nama' => 'required',
        'deksripsi' => 'required',
        'harga' => 'required|numeric'
    ]);

    $product = Product::create($request->all());

    return response()->json([
        'status' => true,
        'message' => 'Product berhasil ditambahkan',
        'data' => $product
    ]);
}
```

Gambar 06: Fungsi Store pada ProductsController

```php
// 🔹 PUT /products/{id}
public function update(Request $request, $id)
{
    $product = Product::find($id);
    if (!$product) {
        return response()->json([
            'status' => false,
            'message' => 'Product tidak ditemukan'
        ], 404);
    }
    $request->validate([
        'nama' => 'required',
        'deksripsi' => 'required',
        'harga' => 'required|numeric'
    ]);
    $product->update($request->all());
    return response()->json([
        'status' => true,
        'message' => 'Product berhasil diupdate',
        'data' => $product
    ]);
}
```

Gambar 07: Fungsi Update pada ProductsController

```php
// 🔹 DELETE /products/{id}
public function destroy($id)
{
    $product = Product::find($id);

    if (!$product) {
        return response()->json([
            'status' => false,
            'message' => 'Product tidak ditemukan'
        ], 404);
    }

    $product->delete();

    return response()->json([
        'status' => true,
        'message' => 'Product berhasil dihapus'
    ]);
}
```

Gambar 08: Fungsi Destroy pada ProductsController

E. API Documentation

1. Install Scramble dengan menggunakan composer
   ```
   composer require dedoc/scramble
   ```

2. Masukan ke vendor laravel project anda
   ```
   php artisan vendor:publish
   --provider="Dedoc\Scramble\ScrambleServiceProvider"
   ```

3. Jalankan server (artisan serve)
4. Akses API documentation pada browser ke http://localhost:8000/docs/api
5. Untuk mendambahkan deksripsi setiap API Endpoints bisa menggunakan DocBlock diatas fungsi dalam controller
6. Menambahkan DocBlock (Title dan Description)

```php
class ProductController extends Controlle
{

/**
 * Ambil semua produk.
 *
 * Endpoint ini mengembalikan seluruh data produk yang tersedia.
 */

    // 🔹 GET /products
    public function index()
    {
        $products = Product::all();

        return response()->json([
            'status' => true,
            'data' => $products
        ]);
    }
```

Gambar 08: DocBlock untuk title node index dan deksripsi

7. Refresh browser ke http://localhost:8000/docs/api
8. Tambahkan Title dan Deskripsi pada semua Endpoints

Hasil Pengerjaan

No.	Instruksi	Screenshot	Kendala/Saran
A.	**Setup laravel dan mysql**		
1.	Update File .env	DB_CONNECTION=mysql DB_HOST=127.0.0.1 DB_PORT=3306 DB_DATABASE=dblaravelapi DB_USERNAME=root DB_PASSWORD=	
2.	Jalankan Migrasi	INFO Running migrations. 0001_01_01_000000_create_users_table ..... 129.68ms DONE 0001_01_01_000001_create_cache_table ..... 74.18ms DONE 0001_01_01_000002_create_jobs_table ..... 77.60ms DONE	
3.	Cek Tabel di MySQL	Tables_in_dblaravelapi cache cache_locks failed_jobs job_batches jobs migrations password_reset_tokens sessions users	
B.	**Membuat Controller**		
1.	Buat Controller ProductController dan tambahkan api	return Application::configure(basePath: dirname(__DIR__)) ->withRouting(web: __DIR__.'/../routes/web.php', api: __DIR__.'/../routes/api.php', commands: __DIR__.'/../routes/console.php', health: '/up',)	
2.	Tambah Route di routes/api.php	routes > api.php 1 <?php 2 3 use Illuminate\Support\Facades\Route; 4 use App\Http\Controllers\Api\ProductController; 5 6 Route::get('/products', [ProductController::class, 'index']);	

3.	Test API	localhost:8000/api/products
		Pretty-print ☑
		{
		"status": true,
		"message": "API bekerja!"
		}	

C.	**Gunakan Database dan Model**		
1.	Buat Tabel tblproducts	INFO Running migrations. 2026_04_06_072812_create_tblproducts_table 32.68ms DONE	

<table>
<tr><td>2.</td><td>Tambahkan 10 Data Produk</td><td>
<table>
<tr><th></th><th>id</th><th>nama</th><th>deskripsi</th><th>foto</th><th>harga</th></tr>
<tr><td>Edit Copy Delete</td><td>1</td><td>Smart TV Samsung 55"</td><td>TV LED 4K UHD dengan fitur Smart TV dan HDR</td><td>tv_samsung.jpg</td><td>7500000.00</td></tr>
<tr><td>Edit Copy Delete</td><td>2</td><td>Kulkas LG 2 Pintu</td><td>Kulkas inverter 2 pintu kapasitas 312 liter</td><td>kulkas_lg.jpg</td><td>5200000.00</td></tr>
<tr><td>Edit Copy Delete</td><td>3</td><td>AC Daikin 1 PK</td><td>AC split inverter hemat energi 1 PK</td><td>ac_daikin.jpg</td><td>4800000.00</td></tr>
<tr><td>Edit Copy Delete</td><td>4</td><td>Mesin Cuci Panasonic</td><td>Mesin cuci front loading 8kg</td><td>mc_panasonic.jpg</td><td>3900000.00</td></tr>
<tr><td>Edit Copy Delete</td><td>5</td><td>Laptop ASUS VivoBook</td><td>Laptop 14 inch Intel Core i5 RAM 8GB SSD 512GB</td><td>laptop_asus.jpg</td><td>8500000.00</td></tr>
<tr><td>Edit Copy Delete</td><td>6</td><td>Smartphone iPhone 15</td><td>iPhone 15 128GB layar 6.1 inch</td><td>iphone15.jpg</td><td>14000000.00</td></tr>
<tr><td>Edit Copy Delete</td><td>7</td><td>Vacuum Cleaner Dyson</td><td>Vacuum cleaner nirkabel dengan daya hisap kuat</td><td>dyson.jpg</td><td>6200000.00</td></tr>
<tr><td>Edit Copy Delete</td><td>8</td><td>Rice Cooker Zojirushi</td><td>Rice cooker digital 1.8 liter dengan timer</td><td>rice_cooker.jpg</td><td>1800000.00</td></tr>
<tr><td>Edit Copy Delete</td><td>9</td><td>Speaker Bluetooth JBL</td><td>Speaker portable waterproof dengan bass kuat</td><td>jbl.jpg</td><td>1200000.00</td></tr>
<tr><td>Edit Copy Delete</td><td>10</td><td>Microwave Sharp</td><td>Microwave 23 liter dengan fitur grill</td><td>microwave_sharp.jpg</td><td>900000.00</td></tr>
</table>
</td><td></td></tr>
<tr><td>3.</td><td>Buat Model Product</td><td>
app > Models > Product.php
<pre>
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Product extends Model
{
    protected $table = 'tblproducts';
    protected $primaryKey = 'id';
    public $timestamps = false;

    protected $fillable = [
        'nama',
        'deskripsi',
        'foto',
        'harga',
    ];
}
</pre>
</td><td></td></tr>
<tr><td>4.</td><td>Update Fungsi index di Controller</td><td>
localhost:8000/api/products
Pretty-print
<pre>
{
  "status": true,
  "data": [
    {
      "id": 1,
      "nama": "Smart TV Samsung 55\"",
      "deskripsi": "TV LED 4K UHD dengan fitur Smart TV dan HDR",
      "foto": "tv_samsung.jpg",
      "harga": "7500000.00"
    },
    {
      "id": 2,
      "nama": "Kulkas LG 2 Pintu",
      "deskripsi": "Kulkas inverter 2 pintu kapasitas 312 liter",
      "foto": "kulkas_lg.jpg",
      "harga": "5200000.00"
    },
    {
      "id": 3,
      "nama": "AC Daikin 1 PK",
      "deskripsi": "AC split inverter hemat energi 1 PK",
      "foto": "ac_daikin.jpg",
      "harga": "4800000.00"
    },
    {
      "id": 4,
      "nama": "Mesin Cuci Panasonic",
      "deskripsi": "Mesin cuci front loading 8kg",
      "foto": "mc_panasonic.jpg",
      "harga": "3900000.00"
    },
    {
      "id": 5,
      "nama": "Laptop ASUS VivoBook",
      "deskripsi": "Laptop 14 inch Intel Core i5 RAM 8GB SSD 512GB",
      "foto": "laptop_asus.jpg",
</pre>
</td><td></td></tr>
<tr><td>D.</td><td>Membuat CRUD standar API Resource</td><td></td><td></td></tr>
<tr><td>1.</td><td>Test CRUD dengan Postman</td><td>Lihat semua produk</td><td></td></tr>
</table>

GET localhost:8000/api/products

Docs Params Authorization Headers (6) Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL

This request does not have a bo

Body Cookies Headers (7) Test Results

{} JSON Preview Visualize


```
{
    "status": true,
    "data": [
        {
            "id": 1,
            "nama": "Smart TV Samsung 55\"",
            "deskripsi": "TV LED 4K UHD dengan fitur Smart TV dan HDR",
            "foto": "tv_samsung.jpg",
            "harga": "7500000.00"
        },
        {
            "id": 2,
            "nama": "Kulkas LG 2 Pintu",
            "deskripsi": "Kulkas inverter 2 pintu kapasitas 312 liter",
            "foto": "kulkas_lg.jpg",
            "harga": "5200000.00"
        },
        {
            "id": 3,
            "nama": "AC Daikin 1 PK",
            "deskripsi": "AC split inverter hemat energi 1 PK",
            "foto": "ac_daikin.jpg",
            "harga": "4800000.00"
        },
```


Lihat produk ID 1

GET localhost:8000/api/products/1

Docs Params Authorization Headers (6) Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL

This request do

Body Cookies Headers (7) Test Results

{} JSON Preview Visualize


```
{
    "status": true,
    "data": {
        "id": 1,
        "nama": "Smart TV Samsung 55\"",
        "deskripsi": "TV LED 4K UHD dengan fitur Smart TV dan HDR",
        "foto": "tv_samsung.jpg",
        "harga": "7500000.00"
    }
}
```


Tambah produk baru

POST localhost:8000/api/products

Docs Params Authorization Headers (10) Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON


```
{
    "nama": "TV Baru",
    "deskripsi": "Deskripsi TV baru",
    "foto": "tv_baru.jpg",
    "harga": 5000000
}
```

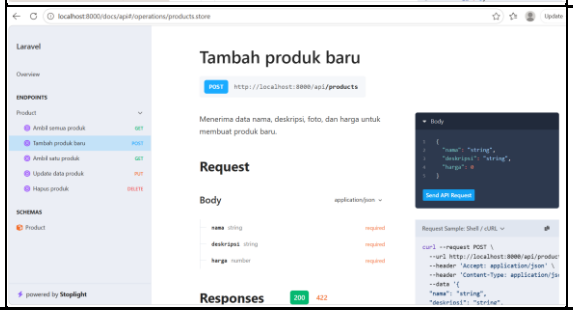

Body Cookies (2) Headers (7) Test Results

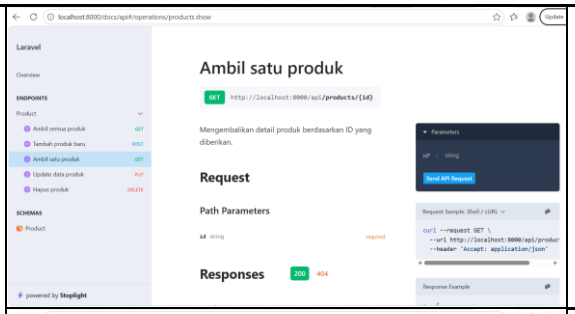
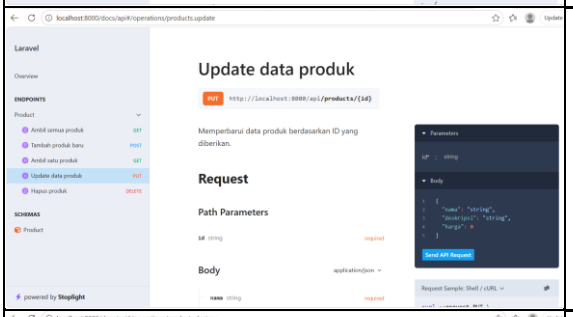
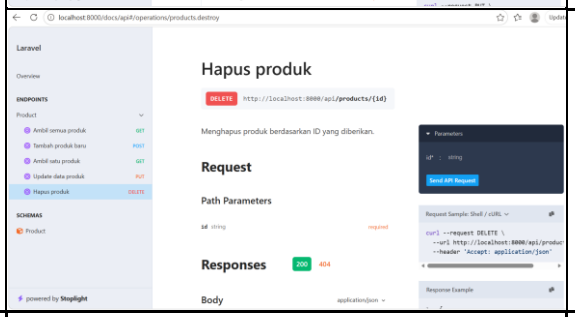
{} JSON Preview Visualize


```
{
    "status": true,
    "message": "Product berhasil ditambahkan",
    "data": {
        "nama": "TV Baru",
        "deskripsi": "Deskripsi TV baru",
        "foto": "tv_baru.jpg",
        "harga": 5000000,
        "id": 11
    }
}
```


Update produk ID 1

		PUT localhost:8000/api/products/1
		Docs Params Authorization Headers (10) Body
		none form-data x-www-form-urlencoded raw
		{
		"nama": "TV Samsung",
		"deskripsi": "Deskripsi TV baru",
		"foto": "tv_baru.jpg",
		"harga": 5000000
		}
		Body Cookies (2) Headers (7) Test Results
		JSON Preview Visualize
		{
		"status": true,
		"message": "Product berhasil diupdate",
		"data": {
		"id": 1,
		"nama": "TV Samsung",
		"deskripsi": "Deskripsi TV baru",
		"foto": "tv_baru.jpg",
		"harga": 5000000
		}
		}
		Hapus produk ID 1
		DELETE localhost:8000/api/products/1
		Docs Params Authorization Headers (7) Body
		none form-data x-www-form-urlencoded
		Body Cookies (2) Headers (7) Test Results
		JSON Preview Visualize
		{
		"status": true,
		"message": "Product berhasil dihapus"
		}	

E.	**API Documentation**		
1.	Akses Dokumentasi	 	

			
			
			
F.	**Link Github**	https://github.com/auliasalsabilla/P05	